

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 06 – Fourier Analysis and the FFT

1

Fourier Analysis

- Period and frequency
- Trigonometric polynomials
- Gibbs phenomenon

2

Complex form

- Discrete Fourier Transform (DFT)
- Fast Fourier Transform (FFT)
- De-trending
- Power spectrum

Fourier analysis

Fourier analysis transforms a signal from the time domain into a frequency-domain representation.

Many signals of interest are periodic, so we first recall the notions of period and frequency.

- A function g has **period** $T > 0$ if $g(t + T) = g(t)$ for all $t \in \mathbb{R}$.
- If g has period T , then it also has periods $2T$, $3T$, $4T$, and so on.
- We say that g has **frequency** (**number of occurrences of a repeating event per unit time**)

$$f = \frac{1}{T}.$$

For instance, a function of the form $\cos \omega t$
($= \cos(\omega t + 2\pi) = \cos \omega(t + 2\pi/\omega)$) has

$$\text{period } T = \frac{2\pi}{\omega} \quad \text{and} \quad \text{frequency } f = \frac{\omega}{2\pi}.$$

We call ω the **angular frequency**.

Trigonometric polynomials

For $k = 1, 2, 3, \dots$, the functions

$$\cos \frac{2\pi kt}{T} = \cos\left(\frac{2\pi kt}{T} + 2\pi\right) = \cos\left(\frac{2\pi k}{T}(t + T/k)\right)$$

and $\sin \frac{2\pi kt}{T}$ have

- period T/k ,
- frequency $f_k = k/T$ (number of occurrences of a repeating event per unit time),
- angular frequency $\omega_k = 2\pi k/T$.

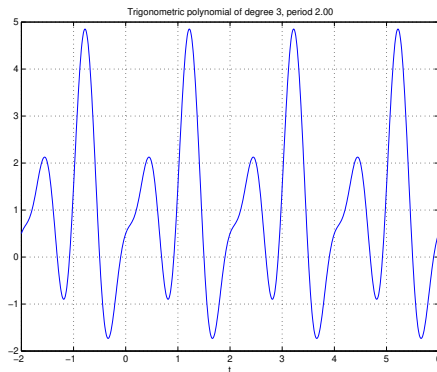
Therefore, a trigonometric polynomial of degree n ,

$$a_0 + \sum_{k=1}^n \left(a_k \cos \frac{2\pi kt}{T} + b_k \sin \frac{2\pi kt}{T} \right),$$

has period T .

Trigonometric polynomial with $T = 2$, $n = 3$ and coefficients

k	0	1	2	3
a_k	1	-1	0	1/2
b_k		0	2	-1



MATLAB `trigpoly.m`, Python file `trigpoly.py`

Orthogonality properties of $\sin(\alpha kt)$ & $\cos(\alpha kt)$, $\alpha = \frac{2\pi}{T}$

$$\begin{aligned}\int_{-T/2}^{T/2} \cos(\alpha kt) \cos(\alpha jt) dt &= 2 \int_0^{T/2} \cos(\alpha kt) \cos(\alpha jt) dt \\ &= \int_0^{T/2} [\cos(\alpha(k-j)t) + \cos(\alpha(k+j)t)] dt = \begin{cases} T/2 & \text{if } j = k \geq 1, \\ 0 & \text{if } j \neq k. \end{cases}\end{aligned}$$

$$\begin{aligned}\int_{-T/2}^{T/2} \sin(\alpha kt) \sin(\alpha jt) dt &= 2 \int_0^{T/2} \sin(\alpha kt) \sin(\alpha jt) dt \\ &= \int_0^{T/2} [\cos(\alpha(k-j)t) - \cos(\alpha(k+j)t)] dt = \begin{cases} T/2 & \text{if } j = k \geq 1, \\ 0 & \text{if } j \neq k. \end{cases}\end{aligned}$$

$$\int_{-T/2}^{T/2} \sin(\alpha kt) \cos(\alpha jt) dt = 0, \quad \text{for all } j, k \geq 1.$$

Computations of a_k and b_k

If

$$g(t) = a_0 + \sum_{k=1}^n \left(a_k \cos \frac{2\pi kt}{T} + b_k \sin \frac{2\pi kt}{T} \right), \quad (1)$$

then how do we find a_0 , a_k and b_k in terms of g ?

- Integrating both sides over the interval $[-T/2, T/2]$,

$$\int_{-T/2}^{T/2} g(t) dt = a_0 T + \sum_{k=1}^n \left(a_k \int_{-T/2}^{T/2} \cos \frac{2\pi kt}{T} dt + b_k \int_{-T/2}^{T/2} \sin \frac{2\pi kt}{T} dt \right),$$

and so,

$$a_0 = \frac{1}{T} \int_{-T/2}^{T/2} g(t) dt, \quad (\text{continuous average value of } g). \quad (2)$$

- Multiplying both sides of (1) by $\cos \frac{2\pi jt}{T}$, integrating over $[-T/2, T/2]$, and using the orthogonality properties, we obtain

$$\begin{aligned} \int_{-T/2}^{T/2} g(t) \cos \frac{2\pi jt}{T} dt &= a_0 \int_{-T/2}^{T/2} \cos \frac{2\pi jt}{T} dt \\ &+ \sum_{k=1}^n \left(a_k \int_{-T/2}^{T/2} \cos \frac{2\pi kt}{T} \cos \frac{2\pi jt}{T} dt + b_k \int_{-T/2}^{T/2} \sin \frac{2\pi kt}{T} \cos \frac{2\pi jt}{T} dt \right) \\ &= \frac{T}{2} a_j. \end{aligned}$$

Therefore,

$$a_k = \frac{2}{T} \int_{-T/2}^{T/2} g(t) \cos \frac{2\pi kt}{T} dt, \quad \text{for } k = 1, 2, \dots, n. \quad (3)$$

Multiplying both sides of (1) by $\sin \frac{2\pi jt}{T}$ and proceeding similarly, we obtain

$$b_k = \frac{2}{T} \int_{-T/2}^{T/2} g(t) \sin \frac{2\pi kt}{T} dt, \quad \text{for } k = 1, 2, \dots, n. \quad (4)$$

Fourier series expansion

For a general function g with period T , the formulae (2)–(4) define the n th partial sum of the Fourier series

$$\mathcal{F}_n g(t) = a_0 + \sum_{k=1}^n \left(a_k \cos \frac{2\pi kt}{T} + b_k \sin \frac{2\pi kt}{T} \right).$$

If g is piecewise differentiable, then

- the Fourier series expansion $\mathcal{F} g(t) = \lim_{n \rightarrow \infty} \mathcal{F}_n g(t)$ converges to $g(t)$ at every t where g is continuous.
- If g has a jump discontinuity at t , then $\mathcal{F} g(t) = [g(t^+) + g(t^-)]/2$.

Consider, as an example,

$$g(t) = \begin{cases} 1 & \text{for } |t| < 1, \\ 0 & \text{for } 1 < |t| < \pi, \end{cases}$$

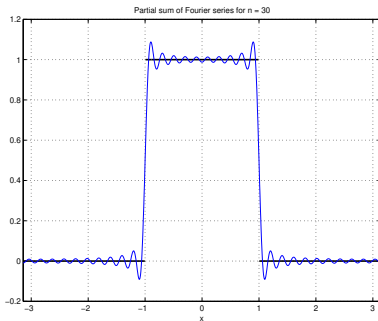
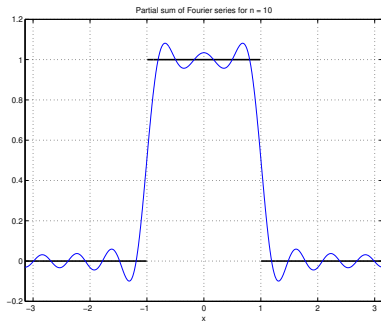
where g has period $T = 2\pi$. We find that

$$a_0 = \frac{1}{2\pi} \int_{-1}^1 dt = \frac{1}{\pi}, \quad a_k = \frac{1}{\pi} \int_{-1}^1 \cos(kt) dt = \frac{2 \sin k}{\pi k}, \quad b_k = \frac{1}{\pi} \int_{-1}^1 \sin(kt) dt = 0.$$

Gibbs phenomenon: oscillatory behavior of the partial sum of a Fourier series near jump discontinuities.

Therefore,

$$\mathcal{F}_n g(t) = \frac{1}{\pi} + \sum_{k=1}^n \frac{2 \sin k}{\pi k} \cos kt.$$



See MATLAB `fsex1.m` or Python `fsex1.py`.

Complex form

If g is continuous at t , the complex form of the Fourier series expansion is

$$g(t) = \mathcal{F} g(t) = \sum_{k=-\infty}^{\infty} c_k e^{i2\pi kt/T} \quad (i^2 = -1).$$

Using the orthogonality property:

$$\int_{-T/2}^{T/2} e^{i2\pi \textcolor{red}{k}t/T} e^{-i2\pi \textcolor{red}{j}t/T} dt = \int_{-T/2}^{T/2} e^{i2\pi(\textcolor{red}{k}-\textcolor{red}{j})t/T} dt = \begin{cases} T & \text{if } k = j, \\ 0 & \text{if } k \neq j, \end{cases}$$

we obtain
$$\int_{-T/2}^{T/2} g(t) e^{-i2\pi jt/T} dt = \sum_{k=-\infty}^{\infty} c_k \int_{-T/2}^{T/2} e^{i2\pi \textcolor{red}{k}t/T} e^{-i2\pi \textcolor{red}{j}t/T} dt = c_j T.$$

Hence,
$$c_k = \frac{1}{T} \int_{-T/2}^{T/2} g(t) e^{-i2\pi kt/T} dt.$$

Using Euler's identity: $e^{iz} = \textcolor{red}{\cos z} + i \textcolor{red}{\sin z}$, one can easily verify that

$$c_0 = a_0, \quad c_k = \frac{a_k - ib_k}{2} \quad \text{and} \quad c_{-k} = \frac{a_k + ib_k}{2} \quad \text{for } k = 1, 2, 3, \dots$$

Discrete Fourier Transform (DFT)

Let $\omega = e^{i2\pi/n}$. Then $\omega^n = e^{i2\pi} = 1$, and so, ω is an n th root of unity. The DFT of $(y_j)_{j=0}^{n-1}$ is

$$\hat{y}_k = \sum_{j=0}^{n-1} y_j e^{-i2\pi jk/n} = \sum_{j=0}^{n-1} y_j \omega^{-jk} \quad \text{for } k = 0, 1, \dots, n-1. \quad (5)$$

Multiply both sides of (5) by $e^{i2\pi \ell k/n} = \omega^{\ell k}$ and sum over k ,

$$\begin{aligned} \sum_{k=0}^{n-1} \omega^{\ell k} \hat{y}_k &= \sum_{k=0}^{n-1} \omega^{\ell k} \sum_{j=0}^{n-1} y_j \omega^{-jk} = \sum_{j=0}^{n-1} y_j \sum_{k=0}^{n-1} (\omega^{\ell-j})^k \\ &= y_\ell \sum_{k=0}^{n-1} 1 + \sum_{j=0, j \neq \ell}^{n-1} y_j \frac{1 - \omega^{(\ell-j)n}}{1 - \omega^{\ell-j}} = ny_\ell. \end{aligned}$$

Therefore, the inverse DFT is

$$y_\ell = \frac{1}{n} \sum_{k=0}^{n-1} \hat{y}_k \omega^{\ell k} = \frac{1}{n} \sum_{k=0}^{n-1} \hat{y}_k e^{i2\pi \ell k/n}, \quad \ell = 0, \dots, n-1.$$

Fast Fourier Transform (FFT)

- Computing $(\hat{y}_k)_{k=0}^{n-1}$ from $(y_j)_{j=0}^{n-1}$, or vice versa, costs $O(n^2)$ operations if we use the obvious approach.
- The **FFT** algorithm reduces this cost to only $O(n \log_2 n)$, and it also reduces the rounding errors.
- The **FFT** algorithm is used to predict future values in many fields from environmental data to financial data.
- The speed and accuracy of the **FFT** are impressive. FFT is one of the top 10 computational algorithms of the 20th century
 - SIAM News: Barry Cipra, “The Best of the 20th Century: Editors Name Top 10 Algorithms”, www.siam.org/pdf/news/637.pdf
 - Nick Trefethen, “Who invented the great numerical algorithms”, people.maths.ox.ac.uk/trefethen/inventorstalk.pdf
- MATLAB-Python functions **fft** and **ifft** carry out the DFT and its inverse, respectively, using the FFT algorithm.

Example (FFT)

On a 3 GHz quad-core PC that can do four flops per clock cycle, estimate how long it will take to compute a DFT with $n = 2^{30}$ using a naive algorithm compared with the FFT.

Solution. A 3 GHz quad-core PC with 4 flops per clock cycle per core can do $3 \times 10^9 \times 4 \times 4 = 48 \times 10^9$ flops/sec.

To compute DFT with $n = 2^{30}$ using a naive algorithm, we need approximately $n^2 = 2^{60} \approx (10^3)^6 = 10^{18}$ flops. Hence the estimated time is

$$\frac{10^{18}}{48 \times 10^9} \approx 2 \times 10^7 \text{ sec} \approx 231 \text{ days}.$$

To compute a DFT with $n = 2^{30}$ using the FFT algorithm, we need approximately $n \log_2 n = 2^{30} \times 30 \approx (10^3)^3 \times 30 = 3 \times 10^{10}$ flops. Hence, the estimated time is

$$\frac{3 \times 10^{10}}{48 \times 10^9} = \frac{10}{16} \approx 0.63 \text{ sec}.$$

Examples on using fft and ifft.

Example 1: `y=randn(4,1)+i*randn(4,1);`

y	fft(y)	ifft(fft(y))
-0.4326 - 1.1465i	-1.6851 + 1.1960i	-0.4326 - 1.1465i
-1.6656 + 1.1909i	0.6707 - 0.3824i	-1.6656 + 1.1909i
0.1253 + 1.1892i	1.0707 - 1.1106i	0.1253 + 1.1892i
0.2877 - 0.0376i	-1.7864 - 4.2889i	0.2877 - 0.0376i

When the y_j are all real, the DFT contains redundant information.Example 2: `y=rand(7,1);`

y	fft(y)
0.4769	2.9946
-2.1504	-2.8651 + 3.2449i
0.4993	0.7096 + 1.8465i
0.7396	2.3272 + 1.4878i
1.5480	2.3272 - 1.4878i
1.1845	0.7096 - 1.8465i
0.6969	-2.8651 - 3.2449i

MATLAB `fftex1.m`, Python `fftex1.py`

Lemma

If $y_0, y_1, \dots, y_{n-1}$ are all real, then $\hat{y}_0$ is real and

$$\hat{y}_{n-k} = \overline{\hat{y}_k}, \quad k = 1, 2, \dots, n-1,$$

where $\overline{\hat{y}_k}$ denotes the complex conjugate of $\hat{y}_k$.

Proof. Recall that $\omega = e^{i2\pi/n}$, so $\omega^n = 1$. Then

$$\hat{y}_{n-k} = \sum_{j=0}^{n-1} y_j e^{-i2\pi j(n-k)/n} = \sum_{j=0}^{n-1} y_j \omega^{-j(n-k)} = \sum_{j=0}^{n-1} y_j \omega^{-jn} \omega^{jk}.$$

Using $\omega^{jn} = (\omega^n)^j = 1$ and $\omega^{jk} = \overline{\omega^{-jk}}$, we have

$$\hat{y}_{n-k} = \sum_{j=0}^{n-1} y_j \overline{\omega^{-jk}} = \overline{\sum_{j=0}^{n-1} y_j \omega^{-jk}} = \overline{\hat{y}_k}. \quad \square$$

Note 1: From (5), we have $\hat{y}_0 = \sum_{j=0}^{n-1} y_j$, is real.

Note 2: If n is even, then $\hat{y}_{n/2}$ is real because $\hat{y}_{n/2} = \overline{\hat{y}_{n/2}}$.

Approximations of Fourier coefficients

Suppose $g(t+T) = g(t)$ and $y_j \approx g(t_j)$, where $t_j = j\Delta t$, $\Delta t = T/n$. We use the DFT of the data y_j to compute approximate Fourier coefficients for g .

Theorem

Suppose we have an even number $n = 2p$ of data points. Define

$$A_k = \frac{2\operatorname{Re}(\hat{y}_k)}{n} \quad \text{and} \quad B_k = -\frac{2\operatorname{Im}(\hat{y}_k)}{n} \quad \text{for } k = 1, 2, \dots, p-1,$$

with

$$A_0 = \frac{\hat{y}_0}{n}, \quad A_p = \frac{\hat{y}_p}{n}, \quad B_p = 0.$$

Then

$$y_j = A_0 + \sum_{k=1}^p \left(A_k \cos \frac{2\pi k t_j}{T} + B_k \sin \frac{2\pi k t_j}{T} \right), \quad \text{for } j = 0, \dots, n-1.$$

Therefore $g(t) \approx A_0 + \sum_{k=1}^p \left(A_k \cos \frac{2\pi k t}{T} + B_k \sin \frac{2\pi k t}{T} \right).$

Proof. The inversion formula for the DFT gives

$$ny_j = \sum_{k=0}^{n-1} \hat{y}_k \omega^{jk} = \sum_{k=0}^{p-1} \hat{y}_k \omega^{jk} + \sum_{k=p}^{n-1} \hat{y}_k \omega^{jk}. \quad (6)$$

Re-indexing the second sum, and using $\omega^n = 1$, $\hat{y}_{n-k} = \overline{\hat{y}_k}$, and $\omega^{-jk} = \overline{\omega^{jk}}$,

$$\sum_{k=p}^{n-1} \hat{y}_k \omega^{jk} = \sum_{k=1}^p \hat{y}_{n-k} \omega^{j(n-k)} = \sum_{k=1}^p \overline{\hat{y}_k} \omega^{-jk} = \sum_{k=1}^{p-1} \overline{\hat{y}_k} \overline{\omega^{jk}} + \hat{y}_p \omega^{-jp}.$$

Inserting this in (6), and using $\omega^{-jp} = e^{-i2\pi jp/n} = e^{-i(j\pi)} = \cos(j\pi)$ and $j = t_j/\Delta t = 2\pi t_j/T$,

$$ny_j = \hat{y}_0 + \sum_{k=1}^{p-1} \left(\hat{y}_k \omega^{jk} + \overline{\hat{y}_k \omega^{jk}} \right) + \hat{y}_p \omega^{-jp} = \hat{y}_0 + \sum_{k=1}^{p-1} 2\operatorname{Re}(\hat{y}_k \omega^{jk}) + \hat{y}_p \cos \frac{2\pi p t_j}{T}.$$

Since $\omega^{jk} = e^{i(2\pi jk/n)} = e^{i(2\pi k t_j/T)} = \cos(2\pi k t_j/T) + i \sin(2\pi k t_j/T)$, we have

$$\operatorname{Re}(\hat{y}_k \omega^{jk}) = \operatorname{Re}(\hat{y}_k) \cos \frac{2\pi k t_j}{T} - \operatorname{Im}(\hat{y}_k) \sin \frac{2\pi k t_j}{T}. \quad \square$$

For an odd number $n = 2p + 1$ of data points, we find again that

$$y_j = A_0 + \sum_{k=1}^p \left(A_k \cos \frac{2\pi kt_j}{T} + B_k \sin \frac{2\pi kt_j}{T} \right),$$

with $A_0 = \widehat{y}_0/n$. Now

$$A_k = \frac{2\operatorname{Re}(\widehat{y}_k)}{n} \quad \text{and} \quad B_k = -\frac{2\operatorname{Im}(\widehat{y}_k)}{n} \quad \text{for } k = 1, 2, \dots, p,$$

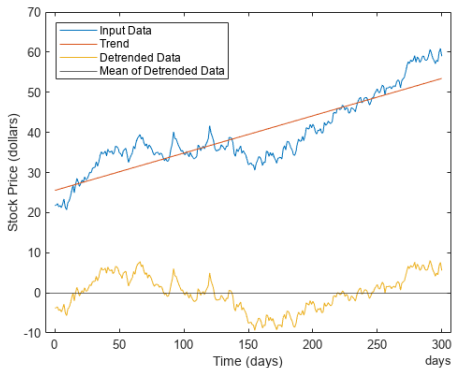
so the coefficients with $k = p$ are not special.

Linear de-trending

Suppose we have data (t_j, Y_j) for $j = 0, 1, \dots, n-1$, with $t_j = j\Delta t$ and $\Delta t = T/n$. For the data Y_j , we suspect periodic fluctuations around a linear trend:

$$Y_j \approx G(t_j), \quad G(t) = x_1 + x_2 t + g(t), \quad g(t+T) = g(t).$$

Graphical example:



Linear de-trending (continued)

Linear de-trending means estimating the linear trend $x_1 + x_2 t$ and then removing it, leaving the periodic component $g(t)$.

We estimate the linear trend by computing the least-squares fit:

$$Y_j \approx x_1 + x_2 t_j.$$

Then we subtract this trend from the data:

$$y_j = Y_j - (x_1 + x_2 t_j) \quad \text{so that} \quad y_j \approx g(t_j).$$

We approximate the de-trended data $y_j \approx g(t_j)$ using a trigonometric polynomial; see the next slide. Then we use the relation

$$G(t) = x_1 + x_2 t + g(t)$$

to estimate $G(t)$.

From trigonometric polynomial approximations,

$$y_j = A_0 + \sum_{k=1}^{\lfloor n/2 \rfloor} \left(A_k \cos \frac{2\pi k t_j}{T} + B_k \sin \frac{2\pi k t_j}{T} \right)$$

for $j = 0, 1, \dots, n-1$, where the approximate Fourier coefficients are

$$A_0 = \hat{y}_0/n, \quad A_k = \frac{2\operatorname{Re}(\hat{y}_k)}{n}, \quad B_k = -\frac{2\operatorname{Im}(\hat{y}_k)}{n}, \quad \text{for } k = 1, \dots, \lfloor n/2 \rfloor - 1,$$

$$A_{n/2} = \frac{\hat{y}_{n/2}}{n} \quad \text{and} \quad B_{n/2} = 0 \quad \text{if } n \text{ is even}$$

$$A_{\lfloor n/2 \rfloor} = \frac{2\operatorname{Re}(\hat{y}_{\lfloor n/2 \rfloor})}{n} \quad \text{and} \quad B_{\lfloor n/2 \rfloor} = -\frac{2\operatorname{Im}(\hat{y}_{\lfloor n/2 \rfloor})}{n} \quad \text{if } n \text{ is odd.}$$

The DFT is computed from the de-trended data $y_j = Y_j - x_1 - x_2 t_j$:

$$\hat{y}_k = \sum_{j=0}^{n-1} y_j e^{-i2\pi jk/n} = \sum_{j=0}^{n-1} (Y_j - x_1 - x_2 t_j) e^{-i2\pi jk/n}.$$

(MATLAB/Python: $\hat{y} = \text{fft}(y)$.) Therefore,

$$g(t) \approx A_0 + \sum_{k=1}^{\lfloor n/2 \rfloor} \left(A_k \cos \frac{2\pi k t}{T} + B_k \sin \frac{2\pi k t}{T} \right).$$

Example: Suppose we have a 10-year time series of monthly real data:
 $T = 10$ and $n = 120$ (n data points).

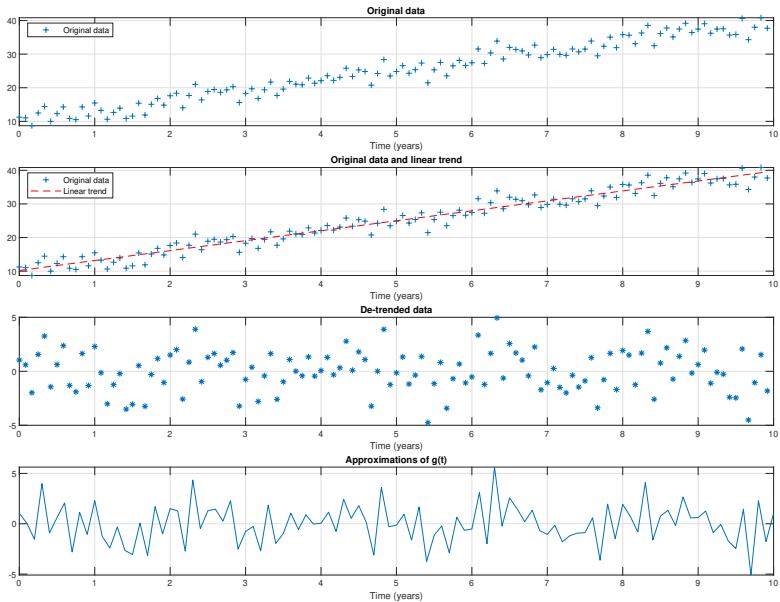
MATLAB script `fftex2.m`

```
T = 10;
n = 120;
Dt = T/n;
t = Dt*[0:n-1]';
Y = ....;
% Linear lsqs approx.
C = [ones(n,1) t];
x = C \ Y;
trend = x(1) + x(2)*t;
% De-trend
y = Y - trend;
yhat = fft(y);
```

Python script `fftex2.py`

```
from numpy import array, arange, ones
from scipy.linalg import lstsq
from scipy.fft import fft
T = 10
n = 120
Dt = T/n
t = Dt * array(arange(n))
Y = ....
C = ones((n, 2))
C[:, 1] = t
x = lstsq(C, Y)[0]
trend = x[0] + x[1] * t
y = Y - trend
yhat = fft(y)
```

The data set Y is generated using the function $10 + 3t + \sin(\pi t) + 2\sin(8\pi t)$ plus random noise `randn(size(t))`.



Power spectrum - dominant periodic components

Using the **dominant periodic components** in $\hat{y}_0, \hat{y}_1, \dots, \hat{y}_{n-1}$, we can approximate the real data $y_0, y_1, \dots, y_{n-1}$ by a smoother signal that captures the main periodic behavior.

Power spectrum: From the DFT coefficients $\hat{y}_k$, compute

$$p_k = \frac{\hat{y}_k \overline{\hat{y}_k}}{n} = \frac{|\hat{y}_k|^2}{n} \quad \text{for } k = 0, 1, \dots, \lfloor n/2 \rfloor.$$

Then plot p_k against the frequency $f_k = k/T$ to find the dominant periodic components in the data.

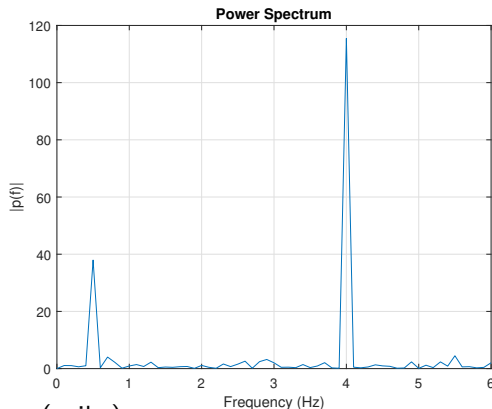
Recall that, for real data y_j , $\hat{y}_{n-k} = \overline{\hat{y}_k}$ for $k = 1, \dots, n-1$. Hence $p_{n-k} = p_k$. Therefore, the coefficients with indices larger than $\lfloor n/2 \rfloor$ are redundant, and it is enough to inspect $k = 0, 1, \dots, \lfloor n/2 \rfloor$.

Example

Suppose we have a 10-year time series of monthly real data:

$$T = 10, \quad n = 120, \quad \Delta t = T/n = 1/12.$$

Power spectrum - dominant periodic components



Large values of p_k (spike) occur at

- $k/T = 0.5 \implies k = 5$, so $|\hat{y}_5|$ and $|\hat{y}_{n-5}|$ are **large**;
- $k/T = 4 \implies k = 40$, so $|\hat{y}_{40}|$ and $|\hat{y}_{n-40}|$ are **large**.

The magnitudes of the other Fourier coefficients are small, so the dominant periodic components correspond to $k = 5$ and $k = 40$.

Power spectrum - dominant periodic components

From the inverse DFT, for $j = 0, \dots, n-1$,

$$\begin{aligned}
 y_j &= \frac{1}{n} \sum_{k=0}^{n-1} \hat{y}_k e^{i2\pi jk/n} \\
 &\approx \frac{1}{n} \left(\hat{y}_5 e^{i2\pi j5/n} + \hat{y}_{40} e^{i2\pi j40/n} + \hat{y}_{n-40} e^{i2\pi j(n-40)/n} + \hat{y}_{n-5} e^{i2\pi j(n-5)/n} \right) \\
 &= \frac{1}{n} \left(\hat{y}_5 e^{i2\pi j5/n} + \hat{y}_{40} e^{i2\pi j40/n} + \overline{\hat{y}_5} e^{-i2\pi j5/n} + \overline{\hat{y}_{40}} e^{-i2\pi j40/n} \right) \\
 &= \frac{1}{n} \left(\hat{y}_5 e^{i2\pi j/24} + \hat{y}_{40} e^{i2\pi j/3} + \overline{\hat{y}_5} e^{i2\pi j/24} + \overline{\hat{y}_{40}} e^{i2\pi j/3} \right) \\
 &= \frac{2}{n} \operatorname{Re} \left(\hat{y}_5 a_j + \hat{y}_{40} b_j \right),
 \end{aligned}$$

where the dominant periodic components:

- $a_j = e^{i2\pi j/24} = e^{i2\pi(j/24+1)} = e^{i2\pi(j+24)/24} = a_{j+24}$, repeats every 2 years,
- $b_j = e^{i2\pi j/3} = e^{i2\pi(j/3+1)} = e^{i2\pi(j+3)/3} = b_{j+3}$, repeats every 3 months.

A comparison between the de-trended data y_j and their approximation using the **two** dominant periodic components.

